

TECHNICAL DOCUMENTATION

Minimal Ledger Service

Production-grade double-entry ledger API — architecture, stack, and source reference

28

Source modules

4

Core API capabilities

1

DB-enforced invariant

Executive Summary

Minimal Ledger Service is a take-home-grade banking core: a HTTP API for creating accounts, posting atomic double-entry transactions, reading balances and statements, and guaranteeing exactly-once writes via idempotency keys.

Correctness is enforced in layers:

- **Database:** deferrable Postgres trigger ensures debits = credits per currency at COMMIT.
- **Application:** row locks (sorted account order), currency checks, optimistic balance versions.
- **Operations:** journal vs materialised balance verifier, Prometheus metrics, structured logs.

Money is represented as `bigint` minor units end-to-end; the journal (`entries`) is append-only and authoritative.

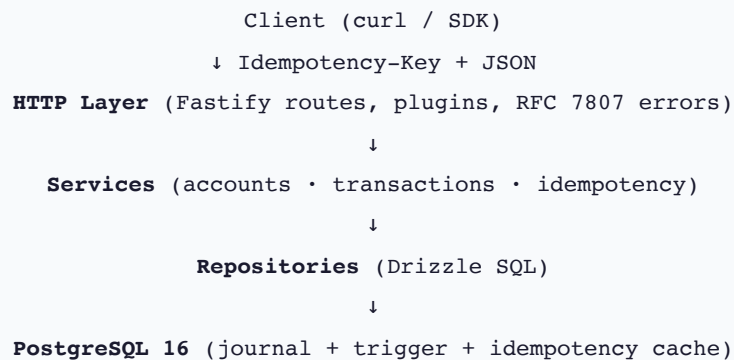


Table of Contents

- | | |
|--------------------------|-------------------|
| 1. Executive Summary | 7. Repositories |
| 2. Tech Stack | 8. Services |
| 3. Project Breakdown | 9. HTTP Layer |
| 4. Entry & Configuration | 10. Jobs |
| 5. Database Layer | 11. Observability |
| 6. Domain Layer | |

Tech Stack

Concern	Technology
Runtime	Node.js 22 LTS
Language	TypeScript 5.x (strict, noUncheckedIndexedAccess)
HTTP	Fastify 5
Database	PostgreSQL 16
ORM / migrations	Drizzle ORM + drizzle-kit
Pool	pg (node-postgres)
Validation	Zod
API docs	@fastify/swagger + swagger-ui (OpenAPI 3.1)
Logging	pino (+ pino-pretty in dev)
Metrics	prom-client
Testing	Vitest, Supertest, Testcontainers, fast-check
Lint / format	ESLint (typescript-eslint strict) + Prettier
IDs	UUID v7 (uuid package)
Money	native bigint (minor units)
Containers	Docker multi-stage + docker-compose

Project Breakdown

Area	Key paths	Responsibility
Entry & config	<code>index.ts, app.ts, config.ts</code>	Boot, Fastify assembly, env validation
Database	<code>db/schema.ts, client.ts, migrate.ts</code>	Drizzle schema, pool, SQL migrations
Domain	<code>domain/*.ts</code>	Money, accounts, posting rules, errors
Repositories	<code>repositories/*.ts</code>	Dumb SQL accessors
Services	<code>services/*.ts</code>	Business orchestration + transactions
HTTP	<code>http/routes, plugins, schemas, error-handler</code>	REST API + idempotency
Jobs	<code>jobs/balance-verification.ts</code>	Journal vs materialised reconciliation
Observability	<code>observability/*.ts</code>	Structured logs + Prometheus
Migrations	<code>migrations/0001_init.sql</code>	Schema + deferred double-entry trigger

Source Code Reference

Full source listings for application code under `src/` and the initial migration. Infrastructure-only directories (e.g. `dist/`, `.github/`, `tests/`) are omitted by design.



Entry & Configuration

src/index.ts

66 lines · Process endpoint

Loads config, runs migrations in development, builds Fastify, schedules idempotency cleanup and optional balance verification, listens on PORT, and implements graceful shutdown on SIGTERM/SIGINT with pool drain.

```
import { loadConfig } from './config.js';
import { createDbContext } from './db/client.js';
import { buildApp } from './app.js';
import { migrate } from './db/migrate.js';
import { startBalanceVerificationScheduler } from './jobs/balance-verification.js';
import { createLogger, getLogger } from './observability/logger.js';
import { IdempotencyRepository } from './repositories/idempotency.repo.js';
import { IdempotencyService } from './services/idempotency.service.js';

async function main(): Promise<void> {
  const config = loadConfig();
  const logger = createLogger(config);
  const dbContext = createDbContext(config);

  if (config.nodeEnv === 'development') {
    await migrate();
  }

  const app = await buildApp({ config, dbContext, logger });

  const idempotencyService = new IdempotencyService(new IdempotencyRepository());
  if (config.idempotencyCleanupIntervalMs > 0) {
    setInterval(() => {
      void idempotencyService.cleanupExpired(dbContext.db, config.idempotencyTtlHours);
    }, config.idempotencyCleanupIntervalMs);
  }

  const balanceScheduler = startBalanceVerificationScheduler(
    dbContext.db,
    config.balanceVerifyIntervalMs,
  );

  await app.listen({ port: config.port, host: config.host });
  logger.info({ port: config.port }, 'Ledger service listening');

  const shutdown = async (signal: string) => {
    logger.info({ signal }, 'Shutting down');
    if (balanceScheduler) {
      clearInterval(balanceScheduler);
    }

    await app.close();
    await dbContext.pool.end();
    logger.info('Shutdown complete');
    process.exit(0);
  };

  const forceExit = setTimeout(() => {
    getLogger().error('Shutdown timeout exceeded');
    process.exit(1);
  }, config.shutdownTimeoutMs);
  forceExit.unref();

  process.on('SIGTERM', () => {
    void shutdown('SIGTERM');
  });
  process.on('SIGINT', () => {
    void shutdown('SIGINT');
  });
}
```

```
main().catch((err: unknown) => {  
  console.error(err);  
  process.exit(1);  
});
```


buildApp() wires Swagger, request context, idempotency plugin, health/metrics routes, and v1 API routes. Records HTTP Prometheus metrics on every response without coupling domain code to HTTP.

```
import { randomUUID } from 'node:crypto';
import Fastify from 'fastify';
import swagger from '@fastify/swagger';
import swaggerUi from '@fastify/swagger-ui';
import type { AppConfig } from './config.js';
import type { DbContext } from './db/client.js';
import { errorHandler } from './http/error-handler.js';
import { idempotencyPlugin } from './http/idempotency.plugin.js';
import { requestContextPlugin } from './http/request-context.plugin.js';
import { accountsRoutes } from './http/routes/accounts.routes.js';
import { healthRoutes } from './http/routes/health.routes.js';
import { transactionsRoutes } from './http/routes/transactions.routes.js';
import { httpRequestDurationSeconds, httpRequestsTotal } from './observability/metrics.js';
import type { Logger } from 'pino';

export type AppDependencies = {
  config: AppConfig;
  dbContext: DbContext;
  logger: Logger;
};

export async function buildApp(deps: AppDependencies) {
  const app = Fastify({
    loggerInstance: deps.logger,
    requestIdHeader: 'x-request-id',
    genReqId: (req) => {
      const header = req.headers['x-request-id'];
      return typeof header === 'string' ? header : randomUUID();
    },
  });

  app.setErrorHandler(errorHandler);

  await app.register(swagger, {
    openapi: {
      openapi: '3.1.0',
      info: {
        title: 'Minimal Ledger Service',
        description: 'Production-grade double-entry ledger API',
        version: '1.0.0',
      },
      servers: [{ url: `http://localhost:${String(deps.config.port)}` }],
    },
  });

  await app.register(swaggerUi, {
    routePrefix: '/docs',
  });

  await app.register(requestContextPlugin);

  app.addHook('onResponse', async (request, reply) => {
    const route = request.routeOptions.url ?? request.url;
    const labels = {
      method: request.method,
      route,
      status: String(reply.statusCode),
    };
    httpRequestsTotal.inc(labels);
    httpRequestDurationSeconds.observe(labels, reply.elapsedTime / 1000);
  });

  await app.register(idempotencyPlugin, { db: deps.dbContext.db });
  await app.register(healthRoutes, { pool: deps.dbContext.pool });
  await app.register(accountsRoutes, { db: deps.dbContext.db });
  await app.register(transactionsRoutes, { db: deps.dbContext.db });
}
```

```
    return app;
  }
}
```

src/config.ts

50 lines · Environment validation

Zod schema over process.env; fails fast at boot if DATABASE_URL or other required settings are invalid. Centralises shutdown timeout, idempotency TTL, and scheduler intervals.

```
import { config as loadDotenv } from 'dotenv';
import { z } from 'zod';

loadDotenv();

const configSchema = z.object({
  NODE_ENV: z.enum(['development', 'test', 'production']).default('development'),
  PORT: z.coerce.number().int().positive().default(3000),
  HOST: z.string().default('0.0.0.0'),
  DATABASE_URL: z.string().url(),
  LOG_LEVEL: z.enum(['fatal', 'error', 'warn', 'info', 'debug', 'trace', 'silent']).default('info'),
  SHUTDOWN_TIMEOUT_MS: z.coerce.number().int().positive().default(25_000),
  IDEMPOTENCY_TTL_HOURS: z.coerce.number().int().positive().default(24),
  IDEMPOTENCY_CLEANUP_INTERVAL_MS: z.coerce.number().int().nonnegative().default(3_600_000),
  BALANCE_VERIFY_INTERVAL_MS: z.coerce.number().int().nonnegative().default(0),
});

export type AppConfig = {
  nodeEnv: 'development' | 'test' | 'production';
  port: number;
  host: string;
  databaseUrl: string;
  logLevel: 'fatal' | 'error' | 'warn' | 'info' | 'debug' | 'trace' | 'silent';
  shutdownTimeoutMs: number;
  idempotencyTtlHours: number;
  idempotencyCleanupIntervalMs: number;
  balanceVerifyIntervalMs: number;
};

export function loadConfig(env: NodeJS.ProcessEnv = process.env): AppConfig {
  const parsed = configSchema.safeParse(env);
  if (!parsed.success) {
    const message = parsed.error.issues.map((i) => `${i.path.join('.')}: ${i.message}`).join('; ');
    throw new Error(`Invalid configuration: ${message}`);
  }

  const e = parsed.data;
  return {
    nodeEnv: e.NODE_ENV,
    port: e.PORT,
    host: e.HOST,
    databaseUrl: e.DATABASE_URL,
    logLevel: e.LOG_LEVEL,
    shutdownTimeoutMs: e.SHUTDOWN_TIMEOUT_MS,
    idempotencyTtlHours: e.IDEMPOTENCY_TTL_HOURS,
    idempotencyCleanupIntervalMs: e.IDEMPOTENCY_CLEANUP_INTERVAL_MS,
    balanceVerifyIntervalMs: e.BALANCE_VERIFY_INTERVAL_MS,
  };
}
```



Database Layer

src/db/schema.ts

120 lines · Drizzle table definitions

Maps Postgres enums and tables: accounts, account_balances, transactions, entries (append-only journal), idempotency_keys. Uses bigint mode for money columns and documents relations for query ergonomics.

```
import { relations, sql } from 'drizzle-orm';
import {
  bigint,
  bigserial,
  char,
  index,
  integer,
  jsonb,
  pgEnum,
  pgTable,
  text,
  timestamp,
  uuid,
} from 'drizzle-orm/pg-core';

export const accountTypeEnum = pgEnum('account_type', [
  'ASSET',
  'LIABILITY',
  'EQUITY',
  'INCOME',
  'EXPENSE',
]);

export const accountStatusEnum = pgEnum('account_status', ['ACTIVE', 'FROZEN', 'CLOSED']);

export const txnStatusEnum = pgEnum('txn_status', ['POSTED', 'REVERSED']);

export const entryDirectionEnum = pgEnum('entry_direction', ['DEBIT', 'CREDIT']);

export const accounts = pgTable('accounts', {
  id: uuid('id').primaryKey(),
  name: text('name').notNull(),
  type: accountTypeEnum('type').notNull(),
  currency: char('currency', { length: 3 }).notNull(),
  status: accountStatusEnum('status').notNull().default('ACTIVE'),
  createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
});

export const accountBalances = pgTable('account_balances', {
  accountId: uuid('account_id')
    .primaryKey()
    .references(() => accounts.id),
  balance: bigint('balance', { mode: 'bigint' })
    .notNull()
    .default(sql`0`),
  version: bigint('version', { mode: 'bigint' })
    .notNull()
    .default(sql`0`),
  updatedAt: timestamp('updated_at', { withTimezone: true }).notNull().defaultNow(),
});

export const transactions = pgTable('transactions', {
  id: uuid('id').primaryKey(),
  status: txnStatusEnum('status').notNull().default('POSTED'),
  description: text('description'),
  externalRef: text('external_ref'),
  postedAt: timestamp('posted_at', { withTimezone: true }).notNull().defaultNow(),
});

export const entries = pgTable(
```

```

    'entries',
    {
      id: bigserial('id', { mode: 'bigint' }).primaryKey(),
      transactionId: uuid('transaction_id')
        .notNull()
        .references(() => transactions.id),
      accountId: uuid('account_id')
        .notNull()
        .references(() => accounts.id),
      direction: entryDirectionEnum('direction').notNull(),
      amount: bigint('amount', { mode: 'bigint' }).notNull(),
      currency: char('currency', { length: 3 }).notNull(),
      postedAt: timestamp('posted_at', { withTimezone: true }).notNull().defaultNow(),
    },
    (table) => [
      index('entries_account_posted_idx').on(table.accountId, table.postedAt),
      index('entries_txn_idx').on(table.transactionId),
    ],
  );

export const idempotencyKeys = pgTable(
  'idempotency_keys',
  {
    key: text('key').primaryKey(),
    requestHash: text('request_hash').notNull(),
    responseStatus: integer('response_status').notNull(),
    responseBody: jsonb('response_body').notNull(),
    createdAt: timestamp('created_at', { withTimezone: true }).notNull().defaultNow(),
  },
  (table) => [index('idempotency_keys_created_at_idx').on(table.createdAt)],
);

export const accountsRelations = relations(accounts, ({ one }) => ({
  balance: one(accountBalances, {
    fields: [accounts.id],
    references: [accountBalances.accountId],
  }),
}));

export const transactionsRelations = relations(transactions, ({ many }) => ({
  entries: many(entries),
}));

export const entriesRelations = relations(entries, ({ one }) => ({
  transaction: one(transactions, {
    fields: [entries.transactionId],
    references: [transactions.id],
  }),
  account: one(accounts, {
    fields: [entries.accountId],
    references: [accounts.id],
  }),
}));

export type AccountRow = typeof accounts.$inferSelect;
export type AccountBalanceRow = typeof accountBalances.$inferSelect;
export type TransactionRow = typeof transactions.$inferSelect;
export type EntryRow = typeof entries.$inferSelect;
export type IdempotencyKeyRow = typeof idempotencyKeys.$inferSelect;

```

Creates pg.Pool and Drizzle Database instance. Exposes checkDbConnectivity() for /readyz readiness probes.

```
import { drizzle, type NodePgDatabase } from 'drizzle-orm/node-postgres';
import pg from 'pg';
import type { AppConfig } from '../config.js';
import * as schema from './schema.js';

export type Database = NodePgDatabase<typeof schema>;

export type DbContext = {
  db: Database;
  pool: pg.Pool;
};

export function createDbContext(config: AppConfig): DbContext {
  const pool = new pg.Pool({
    connectionString: config.databaseUrl,
    max: 20,
  });
  const db = drizzle(pool, { schema });
  return { db, pool };
}

export async function checkDbConnectivity(pool: pg.Pool): Promise<boolean> {
  const client = await pool.connect();
  try {
    await client.query('SELECT 1');
    return true;
  } finally {
    client.release();
  }
}
```

Applies ordered SQL files from migrations/, tracking applied files in schema_migrations. Splits on drizzle-kit statement breakpoints so triggers run as separate statements.

```
import { readFileSync, readdirSync } from 'node:fs';
import { join, dirname } from 'node:path';
import { fileURLToPath } from 'node:url';
import pg from 'pg';
import { loadConfig } from '../config.js';

const __dirname = dirname(fileURLToPath(import.meta.url));
const migrationsDir = join(__dirname, '../migrations');

async function migrate(): Promise<void> {
  const config = loadConfig();
  const pool = new pg.Pool({ connectionString: config.databaseUrl });
  const client = await pool.connect();

  try {
    await client.query(`
      CREATE TABLE IF NOT EXISTS schema_migrations (
        filename TEXT PRIMARY KEY,
        applied_at TIMESTAMPTZ NOT NULL DEFAULT now()
      )
    `);

    const files = readdirSync(migrationsDir)
      .filter((f) => f.endsWith('.sql'))
      .sort();

    for (const file of files) {
      const { rows } = await client.query<{ filename: string }>(
        `SELECT filename FROM schema_migrations WHERE filename = $1`,
        [file],
      );
      if (rows.length > 0) {
        console.log(`skip ${file}`);
        continue;
      }

      const raw = readFileSync(join(migrationsDir, file), 'utf8');
      const statements = raw
        .split('--> statement-breakpoint')
        .map((s) => s.trim())
        .filter((s) => s.length > 0);

      await client.query('BEGIN');
      try {
        for (const statement of statements) {
          await client.query(statement);
        }
        await client.query('INSERT INTO schema_migrations (filename) VALUES ($1)', [file]);
        await client.query('COMMIT');
        console.log(`applied ${file}`);
      } catch (err) {
        await client.query('ROLLBACK');
        throw err;
      }
    }
  } finally {
    client.release();
    await pool.end();
  }
}

const isMain = process.argv[1]?.includes('migrate');
if (isMain) {
  migrate().catch((err: unknown) => {
    console.error(err);
    process.exit(1);
  });
}
```

```
export { migrate };

```

Non-negotiable schema plus `assert_double_entry_balanced()` deferrable constraint trigger — the primary correctness guarantee that debits equal credits per currency at COMMIT.

```
CREATE TYPE "public"."account_type" AS ENUM('ASSET', 'LIABILITY', 'EQUITY', 'INCOME', 'EXPENSE');--> statement-breakpoint
CREATE TYPE "public"."account_status" AS ENUM('ACTIVE', 'FROZEN', 'CLOSED');--> statement-breakpoint
CREATE TYPE "public"."txn_status" AS ENUM('POSTED', 'REVERSED');--> statement-breakpoint
CREATE TYPE "public"."entry_direction" AS ENUM('DEBIT', 'CREDIT');--> statement-breakpoint
CREATE TABLE "accounts" (
    "id" uuid PRIMARY KEY NOT NULL,
    "name" text NOT NULL,
    "type" "account_type" NOT NULL,
    "currency" char(3) NOT NULL,
    "status" "account_status" DEFAULT 'ACTIVE' NOT NULL,
    "created_at" timestamp with time zone DEFAULT now() NOT NULL
);
--> statement-breakpoint
CREATE TABLE "account_balances" (
    "account_id" uuid PRIMARY KEY NOT NULL,
    "balance" bigint DEFAULT 0 NOT NULL,
    "version" bigint DEFAULT 0 NOT NULL,
    "updated_at" timestamp with time zone DEFAULT now() NOT NULL
);
--> statement-breakpoint
CREATE TABLE "transactions" (
    "id" uuid PRIMARY KEY NOT NULL,
    "status" "txn_status" DEFAULT 'POSTED' NOT NULL,
    "description" text,
    "external_ref" text,
    "posted_at" timestamp with time zone DEFAULT now() NOT NULL
);
--> statement-breakpoint
CREATE TABLE "entries" (
    "id" bigserial PRIMARY KEY NOT NULL,
    "transaction_id" uuid NOT NULL,
    "account_id" uuid NOT NULL,
    "direction" "entry_direction" NOT NULL,
    "amount" bigint NOT NULL,
    "currency" char(3) NOT NULL,
    "posted_at" timestamp with time zone DEFAULT now() NOT NULL,
    CONSTRAINT "entries_amount_positive" CHECK ("amount" > 0)
);
--> statement-breakpoint
CREATE TABLE "idempotency_keys" (
    "key" text PRIMARY KEY NOT NULL,
    "request_hash" text NOT NULL,
    "response_status" integer NOT NULL,
    "response_body" jsonb NOT NULL,
    "created_at" timestamp with time zone DEFAULT now() NOT NULL
);
--> statement-breakpoint
ALTER TABLE "account_balances" ADD CONSTRAINT "account_balances_account_id_accounts_id_fk" FOREIGN KEY ("account_id") REFERENCES "public"."accounts"("id") ON DELETE no action ON UPDATE no action;--> statement-breakpoint
ALTER TABLE "entries" ADD CONSTRAINT "entries_transaction_id_transactions_id_fk" FOREIGN KEY ("transaction_id") REFERENCES "public"."transactions"("id") ON DELETE no action ON UPDATE no action;--> statement-breakpoint
ALTER TABLE "entries" ADD CONSTRAINT "entries_account_id_accounts_id_fk" FOREIGN KEY ("account_id") REFERENCES "public"."accounts"("id") ON DELETE no action ON UPDATE no action;--> statement-breakpoint
CREATE INDEX "entries_account_posted_idx" ON "entries" USING btree ("account_id", "posted_at");--> statement-breakpoint
CREATE INDEX "entries_txn_idx" ON "entries" USING btree ("transaction_id");--> statement-breakpoint
CREATE INDEX "idempotency_keys_created_at_idx" ON "idempotency_keys" USING btree ("created_at");

--> statement-breakpoint
CREATE OR REPLACE FUNCTION assert_double_entry_balanced()
RETURNS TRIGGER AS $$
DECLARE
    imbalance RECORD;
BEGIN
    FOR imbalance IN
        SELECT transaction_id, currency,
```



```

        SUM(CASE WHEN direction = 'DEBIT' THEN amount ELSE 0 END) AS debits,
        SUM(CASE WHEN direction = 'CREDIT' THEN amount ELSE 0 END) AS credits
FROM entries
WHERE transaction_id IN (
    SELECT DISTINCT transaction_id FROM new_entries
)
GROUP BY transaction_id, currency
LOOP
    IF imbalance.debits <> imbalance.credits THEN
        RAISE EXCEPTION
            'Double-entry violation: transaction % currency % debits=% credits=%',
            imbalance.transaction_id, imbalance.currency,
            imbalance.debits, imbalance.credits;
    END IF;
END LOOP;
RETURN NULL;
END;
$$ LANGUAGE plpgsql;

--> statement-breakpoint
CREATE CONSTRAINT TRIGGER entries_balanced_check
AFTER INSERT ON entries
DEFERRABLE INITIALLY DEFERRED
REFERENCING NEW TABLE AS new_entries
FOR EACH STATEMENT
EXECUTE FUNCTION assert_double_entry_balanced();

```



Domain Layer

src/domain/money.ts

64 lines · Currency & amount primitives

Supported ISO currencies (USD, EUR, GBP, AED), parse/format of minor-unit bigint amounts from wire strings, and safe add/subtract helpers. No floats anywhere.

```
import { InvalidAmountError, UnsupportedCurrencyError } from './errors.js';

export const CURRENCIES = {
  USD: { code: 'USD', minorUnits: 2 },
  EUR: { code: 'EUR', minorUnits: 2 },
  GBP: { code: 'GBP', minorUnits: 2 },
  AED: { code: 'AED', minorUnits: 2 },
} as const;

export type CurrencyCode = keyof typeof CURRENCIES;

export function isSupportedCurrency(code: string): code is CurrencyCode {
  return code in CURRENCIES;
}

export function assertSupportedCurrency(code: string): CurrencyCode {
  if (!isSupportedCurrency(code)) {
    throw new UnsupportedCurrencyError(code);
  }
  return code;
}

/** Parse wire-format minor-unit amount (integer string) to bigint. */
export function parseAmount(value: string): bigint {
  if (!/^-?\d+$/.test(value)) {
    throw new InvalidAmountError(`Amount must be an integer string: ${value}`);
  }
  try {
    const amount = BigInt(value);
    if (amount <= 0n) {
      throw new InvalidAmountError('Amount must be positive');
    }
    return amount;
  } catch {
    throw new InvalidAmountError(`Amount out of range: ${value}`);
  }
}

export function formatAmount(amount: bigint): string {
  return amount.toString();
}

export function addAmounts(a: bigint, b: bigint): bigint {
  const result = a + b;
  if (result < a && b > 0n) {
    throw new InvalidAmountError('Amount overflow on addition');
  }
  if (result > a && b < 0n) {
    throw new InvalidAmountError('Amount underflow on addition');
  }
  return result;
}

export function subtractAmounts(a: bigint, b: bigint): bigint {
  const result = a - b;
  if (result > a && b > 0n) {
    throw new InvalidAmountError('Amount underflow on subtraction');
  }
  if (result < a && b < 0n) {
    throw new InvalidAmountError('Amount overflow on subtraction');
  }
}
```

```
    return result;  
}
```

Normal-balance rules (DEBIT-normal for ASSET/EXPENSE, CREDIT-normal otherwise), journal balance derivation, and applying entries to materialised balances.

```
import type { AccountRow } from '../db/schema.js';
import { addAmounts, subtractAmounts } from './money.js';

export type AccountType = AccountRow['type'];
export type AccountStatus = AccountRow['status'];
export type EntryDirection = 'DEBIT' | 'CREDIT';

export type Account = {
  id: string;
  name: string;
  type: AccountType;
  currency: string;
  status: AccountStatus;
  createdAt: Date;
};

export function mapAccountRow(row: AccountRow): Account {
  return {
    id: row.id,
    name: row.name,
    type: row.type,
    currency: row.currency.trim(),
    status: row.status,
    createdAt: row.createdAt,
  };
}

/** Normal-balance side: DEBIT for ASSET/EXPENSE, CREDIT for LIABILITY/EQUITY/INCOME. */
export function normalBalanceSide(type: AccountType): EntryDirection {
  return type === 'ASSET' || type === 'EXPENSE' ? 'DEBIT' : 'CREDIT';
}

/**
 * Signed balance from journal aggregates (debits and credits in minor units).
 */
export function balanceFromJournal(type: AccountType, debits: bigint, credits: bigint): bigint {
  if (normalBalanceSide(type) === 'DEBIT') {
    return subtractAmounts(debits, credits);
  }
  return subtractAmounts(credits, debits);
}

/**
 * Apply an entry to a materialised balance using account normal-balance rules.
 */
export function applyEntryToBalance(
  type: AccountType,
  currentBalance: bigint,
  direction: EntryDirection,
  amount: bigint,
): bigint {
  const isDebitNormal = normalBalanceSide(type) === 'DEBIT';
  const increasesBalance =
    (direction === 'DEBIT' && isDebitNormal) || (direction === 'CREDIT' && !isDebitNormal);

  if (increasesBalance) {
    return addAmounts(currentBalance, amount);
  }
  return subtractAmounts(currentBalance, amount);
}

export function assertAccountActive(account: Account): void {
  if (account.status !== 'ACTIVE') {
    throw new Error(`Account ${account.id} is not active`);
  }
}
```

App-side per-currency double-entry check before INSERT; complements the database trigger with friendlier 422 errors.

```
import type { EntryDirection } from './account.js';
import { UnbalancedTransactionError } from './errors.js';
import { addAmounts, type CurrencyCode } from './money.js';

export type PostingEntryInput = {
  accountId: string;
  direction: EntryDirection;
  amount: bigint;
  currency: CurrencyCode;
};

export type PostTransactionInput = {
  description?: string | undefined;
  externalRef?: string | undefined;
  entries: PostingEntryInput[];
};

/**
 * App-side double-entry check per currency before hitting the deferred DB trigger.
 */
export function assertTransactionBalanced(entries: PostingEntryInput[]): void {
  const totals = new Map<string, { debits: bigint; credits: bigint }>();

  for (const entry of entries) {
    const key = entry.currency;
    const bucket = totals.get(key) ?? { debits: 0n, credits: 0n };
    if (entry.direction === 'DEBIT') {
      bucket.debits = addAmounts(bucket.debits, entry.amount);
    } else {
      bucket.credits = addAmounts(bucket.credits, entry.amount);
    }
    totals.set(key, bucket);
  }

  for (const [currency, { debits, credits }] of totals) {
    if (debits !== credits) {
      throw new UnbalancedTransactionError(currency);
    }
  }
}
```

Typed errors (AccountNotFound, UnbalancedTransaction, IdempotencyConflict, etc.) with stable codes. Only the HTTP error handler maps these to RFC 7807 problem+json.

```
export abstract class DomainError extends Error {
  abstract readonly code: string;

  constructor(message: string) {
    super(message);
    this.name = this.constructor.name;
  }
}

export class AccountNotFoundError extends DomainError {
  readonly code = 'account_not_found';

  constructor(accountId: string) {
    super(`Account not found: ${accountId}`);
  }
}

export class AccountFrozenError extends DomainError {
  readonly code = 'account_frozen';

  constructor(accountId: string) {
    super(`Account is frozen: ${accountId}`);
  }
}

export class UnbalancedTransactionError extends DomainError {
  readonly code = 'unbalanced_transaction';

  constructor(currency: string) {
    super(`Transaction is unbalanced for currency ${currency}`);
  }
}

export class CurrencyMismatchError extends DomainError {
  readonly code = 'currency_mismatch';

  constructor(accountId: string, expected: string, actual: string) {
    super(
      `Currency mismatch for account ${accountId}: account currency ${expected}, entry currency ${actual}`
    );
  }
}

export class UnsupportedCurrencyError extends DomainError {
  readonly code = 'unsupported_currency';

  constructor(currency: string) {
    super(`Unsupported currency: ${currency}`);
  }
}

export class IdempotencyConflictError extends DomainError {
  readonly code = 'idempotency_conflict';

  constructor() {
    super('Idempotency key was used with a different request body');
  }
}

export class IdempotencyKeyRequiredError extends DomainError {
  readonly code = 'idempotency_key_required';

  constructor() {
    super('Idempotency-Key header is required for POST requests');
  }
}

export class TransactionNotFoundError extends DomainError {
```

```
readonly code = 'transaction_not_found';

constructor(transactionId: string) {
  super(`Transaction not found: ${transactionId}`);
}
}

export class DoubleEntryViolationError extends DomainError {
  readonly code = 'double_entry_violation';

  constructor(message: string) {
    super(message);
  }
}

export class InvalidAmountError extends DomainError {
  readonly code = 'invalid_amount';

  constructor(message: string) {
    super(message);
  }
}
```



Repositories

src/repositories/accounts.repo.ts

84 lines · Account & balance persistence

Insert/find accounts, SELECT FOR UPDATE in sorted order (deadlock prevention), optimistic versioned balance updates.

```
import { eq, inArray, sql } from 'drizzle-orm';
import type { NodePgDatabase } from 'drizzle-orm/node-postgres';
import type * as schema from '../db/schema.js';
import { accountBalances, accounts } from '../db/schema.js';

export type DbExecutor = NodePgDatabase<typeof schema>;

export class AccountsRepository {
  async insertAccount(
    db: DbExecutor,
    row: typeof accounts.$inferInsert,
  ): Promise<typeof accounts.$inferSelect> {
    const [created] = await db.insert(accounts).values(row).returning();
    if (!created) {
      throw new Error('Failed to insert account');
    }
    return created;
  }

  async insertBalance(db: DbExecutor, row: typeof accountBalances.$inferInsert): Promise<void> {
    await db.insert(accountBalances).values(row);
  }

  async findById(
    db: DbExecutor,
    accountId: string,
  ): Promise<typeof accounts.$inferSelect | undefined> {
    const [row] = await db.select().from(accounts).where(eq(accounts.id, accountId)).limit(1);
    return row;
  }

  async findByIdsForUpdate(
    db: DbExecutor,
    accountIds: string[],
  ): Promise<(typeof accounts.$inferSelect)[]> {
    if (accountIds.length === 0) {
      return [];
    }
    return db
      .select()
      .from(accounts)
      .where(inArray(accounts.id, accountIds))
      .orderBy(accounts.id)
      .for('update');
  }

  async getBalance(
    db: DbExecutor,
    accountId: string,
  ): Promise<typeof accountBalances.$inferSelect | undefined> {
    const [row] = await db
      .select()
      .from(accountBalances)
      .where(eq(accountBalances.accountId, accountId))
      .limit(1);
    return row;
  }

  async updateBalance(
    db: DbExecutor,
    accountId: string,
    balance: bigint,
  ) {
  }
```



```
    expectedVersion: bigint,
  ): Promise<boolean> {
    const result = await db
      .update(accountBalances)
      .set({
        balance,
        version: expectedVersion + 1n,
        updatedAt: sql`now()` ,
      })
      .where(
        sql`${accountBalances.accountId} = ${accountId} AND ${accountBalances.version} = ${expectedVersion}`,
      )
      .returning();
    return result.length > 0;
  }

  async listAllAccountIds(db: DbExecutor): Promise<string[]> {
    const rows = await db.select({ id: accounts.id }).from(accounts);
    return rows.map((r) => r.id);
  }
}
```

Append-only entry inserts, statement pagination, and journal SUM aggregates for verification.
No UPDATE/DELETE on entries.

```
import { and, desc, eq, sql } from 'drizzle-orm';
import { entries } from '../db/schema.js';
import type { DbExecutor } from '../accounts.repo.js';

export class EntriesRepository {
  async insertMany(db: DbExecutor, rows: (typeof entries.$inferInsert)[]): Promise<void> {
    if (rows.length > 0) {
      await db.insert(entries).values(rows);
    }
  }

  async findByTransactionId(
    db: DbExecutor,
    transactionId: string,
  ): Promise<(typeof entries.$inferSelect)[]> {
    return db
      .select()
      .from(entries)
      .where(eq(entries.transactionId, transactionId))
      .orderBy(entries.id);
  }

  async listByAccount(
    db: DbExecutor,
    accountId: string,
    limit: number,
    cursor?: { postedAt: Date; id: bigint },
  ): Promise<(typeof entries.$inferSelect)[]> {
    const conditions = [eq(entries.accountId, accountId)];
    if (cursor) {
      conditions.push(
        sql`${entries.postedAt}, ${entries.id} < (${cursor.postedAt}, ${cursor.id})`,
      );
    }

    return db
      .select()
      .from(entries)
      .where(and(...conditions))
      .orderBy(desc(entries.postedAt), desc(entries.id))
      .limit(limit);
  }

  async sumJournalByAccount(
    db: DbExecutor,
    accountId: string,
  ): Promise<{ debits: bigint; credits: bigint }> {
    const [row] = await db
      .select({
        debits: sql<bigint>`COALESCE(SUM(CASE WHEN ${entries.direction} = 'DEBIT' THEN ${entries.amount}
ELSE 0 END), 0)`,
        credits: sql<bigint>`COALESCE(SUM(CASE WHEN ${entries.direction} = 'CREDIT' THEN ${entries.amoun
t} ELSE 0 END), 0)`,
      })
      .from(entries)
      .where(eq(entries.accountId, accountId));

    return {
      debits: coerceBigInt(row?.debits),
      credits: coerceBigInt(row?.credits),
    };
  }
}

function coerceBigInt(value: bigint | string | number | null | undefined): bigint {
  if (value === null || value === undefined) {
    return 0n;
  }
  if (typeof value === 'bigint') {

```

```
    return value;
  }
  return BigInt(value);
}
```

src/repositories/transactions.repo.ts

29 lines · Transaction headers

Insert and fetch logical transaction rows grouping journal lines.

```
import { eq } from 'drizzle-orm';
import { transactions } from '../db/schema.js';
import type { DbExecutor } from './accounts.repo.js';

export class TransactionsRepository {
  async insert(
    db: DbExecutor,
    row: typeof transactions.$inferInsert,
  ): Promise<typeof transactions.$inferSelect> {
    const [created] = await db.insert(transactions).values(row).returning();
    if (!created) {
      throw new Error('Failed to insert transaction');
    }
    return created;
  }

  async findById(
    db: DbExecutor,
    transactionId: string,
  ): Promise<typeof transactions.$inferSelect | undefined> {
    const [row] = await db
      .select()
      .from(transactions)
      .where(eq(transactions.id, transactionId))
      .limit(1);
    return row;
  }
}
```

Lookup/insert cached responses by client key; TTL cleanup by created_at.

```
import { eq, lt } from 'drizzle-orm';
import { idempotencyKeys } from '../db/schema.js';
import type { DbExecutor } from './accounts.repo.js';

export type IdempotencyRecord = {
  key: string;
  requestHash: string;
  responseStatus: number;
  responseBody: unknown;
};

export class IdempotencyRepository {
  async findByKey(db: DbExecutor, key: string): Promise<IdempotencyRecord | undefined> {
    const [row] = await db
      .select()
      .from(idempotencyKeys)
      .where(eq(idempotencyKeys.key, key))
      .limit(1);
    if (!row) {
      return undefined;
    }
    return {
      key: row.key,
      requestHash: row.requestHash,
      responseStatus: row.responseStatus,
      responseBody: row.responseBody,
    };
  }

  async insert(
    db: DbExecutor,
    record: {
      key: string;
      requestHash: string;
      responseStatus: number;
      responseBody: unknown;
    },
  ): Promise<void> {
    await db.insert(idempotencyKeys).values({
      key: record.key,
      requestHash: record.requestHash,
      responseStatus: record.responseStatus,
      responseBody: record.responseBody,
    });
  }

  async deleteOlderThan(db: DbExecutor, cutoff: Date): Promise<number> {
    const result = await db
      .delete(idempotencyKeys)
      .where(lt(idempotencyKeys.createdAt, cutoff))
      .returning({ key: idempotencyKeys.key });
    return result.length;
  }
}
```

Services

src/services/accounts.service.ts

118 lines · Account lifecycle

createAccount in a DB transaction (account + zero balance row + optional idempotency record).
getBalance returns materialised and journal balances for audit.

```
import { v7 as uuidv7 } from 'uuid';
import type { Database } from '../db/client.js';
import { balanceFromJournal, mapAccountRow, type Account } from '../domain/account.js';
import { assertSupportedCurrency } from '../domain/money.js';
import { AccountNotFoundError } from '../domain/errors.js';
import { AccountsRepository } from '../repositories/accounts.repo.js';
import { EntriesRepository } from '../repositories/entries.repo.js';
import type { IdempotencyContext } from '../idempotency.service.js';
import { IdempotencyService } from '../idempotency.service.js';

export type CreateAccountInput = {
  name: string;
  type: Account['type'];
  currency: string;
};

export type AccountResponse = {
  id: string;
  name: string;
  type: Account['type'];
  currency: string;
  status: Account['status'];
  created_at: string;
};

export type BalanceResponse = {
  account_id: string;
  currency: string;
  balance: string;
  journal_balance: string;
  as_of: string;
};

export class AccountsService {
  constructor(
    private readonly db: Database,
    private readonly accountsRepo: AccountsRepository,
    private readonly entriesRepo: EntriesRepository,
    private readonly idempotencyService: IdempotencyService,
  ) {}

  toAccountResponse(account: Account): AccountResponse {
    return {
      id: account.id,
      name: account.name,
      type: account.type,
      currency: account.currency,
      status: account.status,
      created_at: account.createdAt.toISOString(),
    };
  }

  async createAccount(
    input: CreateAccountInput,
    idempotency?: IdempotencyContext,
    responseForCache?: AccountResponse,
  ): Promise<AccountResponse> {
    const currency = assertSupportedCurrency(input.currency);
    const accountId = uuidv7();

    return this.db.transaction(async (tx) => {
      const row = await this.accountsRepo.insertAccount(tx, {

```

```

        id: accountId,
        name: input.name,
        type: input.type,
        currency,
        status: 'ACTIVE',
    });
    await this.accountsRepo.insertBalance(tx, {
        accountId,
        balance: 0n,
        version: 0n,
    });

    const response = this.toAccountResponse(mapAccountRow(row));

    if (idempotency) {
        await this.idempotencyService.persist(tx, idempotency, 201, responseForCache ?? response);
    }

    return response;
    });
}

async getAccount(accountId: string): Promise<Account> {
    const row = await this.accountsRepo.findById(this.db, accountId);
    if (!row) {
        throw new AccountNotFoundError(accountId);
    }
    return mapAccountRow(row);
}

async getBalance(accountId: string): Promise<BalanceResponse> {
    const account = await this.getAccount(accountId);
    const materialised = await this.accountsRepo.getBalance(this.db, accountId);
    if (!materialised) {
        throw new AccountNotFoundError(accountId);
    }
}

const { debits, credits } = await this.entriesRepo.sumJournalByAccount(this.db, accountId);
const journalBalance = balanceFromJournal(account.type, debits, credits);

return {
    account_id: accountId,
    currency: account.currency,
    balance: materialised.balance.toString(),
    journal_balance: journalBalance.toString(),
    as_of: materialised.updatedAt.toISOString(),
};
}

/** Used by balance verification job – compares materialised vs journal. */
async computeJournalBalance(accountId: string, type: Account['type']): Promise<bigint> {
    const { debits, credits } = await this.entriesRepo.sumJournalByAccount(this.db, accountId);
    return balanceFromJournal(type, debits, credits);
}
}

```

Single Postgres transaction: lock accounts, validate ACTIVE/currency, insert transaction + entries, update materialised balances, persist idempotency cache, increment ledger metrics. COMMIT fires deferred balance trigger.

```
import { v7 as uuidv7 } from 'uuid';
import type { Database } from '../db/client.js';
import { applyEntryToBalance, mapAccountRow, type EntryDirection } from '../domain/account.js';
import {
  AccountFrozenError,
  AccountNotFoundError,
  CurrencyMismatchError,
  TransactionNotFoundError,
} from '../domain/errors.js';
import { assertSupportedCurrency, formatAmount, parseAmount } from '../domain/money.js';
import {
  assertTransactionBalanced,
  type PostingEntryInput,
  type PostTransactionInput,
} from '../domain/transaction.js';
import { AccountsRepository } from '../repositories/accounts.repo.js';
import { EntriesRepository } from '../repositories/entries.repo.js';
import { TransactionsRepository } from '../repositories/transactions.repo.js';
import {
  ledgerEntriesPostedTotal,
  ledgerTransactionsPostedTotal,
} from '../observability/metrics.js';
import type { IdempotencyContext } from './idempotency.service.js';
import { IdempotencyService } from './idempotency.service.js';

export type EntryResponse = {
  id: string;
  transaction_id: string;
  account_id: string;
  direction: EntryDirection;
  amount: string;
  currency: string;
  posted_at: string;
};

export type TransactionResponse = {
  id: string;
  status: 'POSTED' | 'REVERSED';
  description: string | null;
  external_ref: string | null;
  posted_at: string;
  entries: EntryResponse[];
};

export class TransactionsService {
  constructor(
    private readonly db: Database,
    private readonly accountsRepo: AccountsRepository,
    private readonly entriesRepo: EntriesRepository,
    private readonly transactionsRepo: TransactionsRepository,
    private readonly idempotencyService: IdempotencyService,
  ) {}

  async postTransaction(
    input: PostTransactionInput,
    idempotency?: IdempotencyContext,
    responseForCache?: TransactionResponse,
  ): Promise<TransactionResponse> {
    assertTransactionBalanced(input.entries);

    const distinctAccountIds = [...new Set(input.entries.map((e) => e.accountId))].sort();

    const result = await this.db.transaction(async (tx) => {
      const lockedRows = await this.accountsRepo.findByIdsForUpdate(tx, distinctAccountIds);
      const accountMap = new Map(lockedRows.map((r) => [r.id, mapAccountRow(r)]));

      for (const accountId of distinctAccountIds) {
        const account = accountMap.get(accountId);
```

```

    if (!account) {
      throw new AccountNotFoundError(accountId);
    }
    if (account.status === 'FROZEN') {
      throw new AccountFrozenError(accountId);
    }
    if (account.status !== 'ACTIVE') {
      throw new AccountFrozenError(accountId);
    }
  }
}

for (const entry of input.entries) {
  const account = accountMap.get(entry.accountId);
  if (!account) {
    throw new AccountNotFoundError(entry.accountId);
  }
  if (account.currency !== entry.currency) {
    throw new CurrencyMismatchError(entry.accountId, account.currency, entry.currency);
  }
}

const balanceDeltas = new Map<string, bigint>();
for (const entry of input.entries) {
  const account = accountMap.get(entry.accountId);
  if (!account) {
    continue;
  }
  const balanceRow = await this.accountsRepo.getBalance(tx, entry.accountId);
  const current = balanceRow?.balance ?? 0n;
  const deltaKey = entry.accountId;
  const running = balanceDeltas.get(deltaKey) ?? current;
  const next = applyEntryToBalance(account.type, running, entry.direction, entry.amount);
  balanceDeltas.set(deltaKey, next);
}

const transactionId = uuidv7();
const txn = await this.transactionsRepo.insert(tx, {
  id: transactionId,
  status: 'POSTED',
  description: input.description ?? null,
  externalRef: input.externalRef ?? null,
});

const entryRows = input.entries.map((e) => ({
  transactionId,
  accountId: e.accountId,
  direction: e.direction,
  amount: e.amount,
  currency: e.currency,
})));

await this.entriesRepo.insertMany(tx, entryRows);

for (const [accountId, newBalance] of balanceDeltas) {
  const balanceRow = await this.accountsRepo.getBalance(tx, accountId);
  if (!balanceRow) {
    throw new AccountNotFoundError(accountId);
  }
  const updated = await this.accountsRepo.updateBalance(
    tx,
    accountId,
    newBalance,
    balanceRow.version,
  );
  if (!updated) {
    throw new Error(`Optimistic concurrency conflict on account ${accountId}`);
  }
}

const insertedEntries = await this.entriesRepo.findByTransactionId(tx, transactionId);
const response: TransactionResponse = {
  id: txn.id,
  status: txn.status,
  description: txn.description,
  external_ref: txn.externalRef,
  posted_at: txn.postedAt.toISOString(),
  entries: insertedEntries.map((e) => ({
    id: e.id.toString(),
    transaction_id: e.transactionId,

```



```

        account_id: e.accountId,
        direction: e.direction,
        amount: formatAmount(e.amount),
        currency: e.currency.trim(),
        posted_at: e.postedAt.toISOString(),
    })),
    });

    if (idempotency) {
        await this.idempotencyService.persist(tx, idempotency, 201, responseForCache ?? response);
    }

    ledgerTransactionsPostedTotal.inc();
    ledgerEntriesPostedTotal.inc(input.entries.length);

    return response;
    });

    return result;
}

async getTransaction(transactionId: string): Promise<TransactionResponse> {
    const txn = await this.transactionsRepo.findById(this.db, transactionId);
    if (!txn) {
        throw new TransactionNotFoundError(transactionId);
    }
    const entryRows = await this.entriesRepo.findByTransactionId(this.db, transactionId);
    return {
        id: txn.id,
        status: txn.status,
        description: txn.description,
        external_ref: txn.externalRef,
        posted_at: txn.postedAt.toISOString(),
        entries: entryRows.map((e) => ({
            id: e.id.toString(),
            transaction_id: e.transactionId,
            account_id: e.accountId,
            direction: e.direction,
            amount: formatAmount(e.amount),
            currency: e.currency.trim(),
            posted_at: e.postedAt.toISOString(),
        })),
    };
}

static parsePostingEntries(
    entries: {
        account_id: string;
        direction: EntryDirection;
        amount: string;
        currency: string;
    }[],
): PostingEntryInput[] {
    return entries.map((e) => ({
        accountId: e.account_id,
        direction: e.direction,
        amount: parseAmount(e.amount),
        currency: assertSupportedCurrency(e.currency),
    }));
}
}

```

Canonical JSON hashing (sorted keys), cache lookup, conflict detection, and persist-in-transaction API used by services.

```
import { createHash } from 'node:crypto';
import type { IdempotencyRepository } from '../repositories/idempotency.repo.js';
import type { DbExecutor } from '../repositories/accounts.repo.js';
import { IdempotencyConflictError } from '../domain/errors.js';

export type IdempotencyContext = {
  key: string;
  requestHash: string;
};

export type CachedIdempotencyResponse = {
  status: number;
  body: unknown;
};

/** Canonical JSON: sorted keys recursively, no whitespace. */
export function canonicalize(value: unknown): string {
  if (value === null || typeof value !== 'object') {
    return JSON.stringify(value);
  }
  if (Array.isArray(value)) {
    return `[${value.map((v) => canonicalize(v)).join(',')}]`;
  }
  const obj = value as Record<string, unknown>;
  const keys = Object.keys(obj).sort();
  const parts = keys.map((k) => `${JSON.stringify(k)}:${canonicalize(obj[k])}`);
  return `{${parts.join(',')}`;
}

export function hashRequestBody(body: unknown): string {
  return createHash('sha256').update(canonicalize(body)).digest('hex');
}

export class IdempotencyService {
  constructor(private readonly repo: IdempotencyRepository) {}

  async findCached(
    db: DbExecutor,
    ctx: IdempotencyContext,
  ): Promise<CachedIdempotencyResponse | 'conflict' | undefined> {
    const existing = await this.repo.findByKey(db, ctx.key);
    if (!existing) {
      return undefined;
    }
    if (existing.requestHash !== ctx.requestHash) {
      return 'conflict';
    }
    return { status: existing.responseStatus, body: existing.responseBody };
  }

  assertNoConflict(existing: CachedIdempotencyResponse | 'conflict' | undefined): void {
    if (existing === 'conflict') {
      throw new IdempotencyConflictError();
    }
  }

  async persist(
    db: DbExecutor,
    ctx: IdempotencyContext,
    status: number,
    body: unknown,
  ): Promise<void> {
    await this.repo.insert(db, {
      key: ctx.key,
      requestHash: ctx.requestHash,
      responseStatus: status,
      responseBody: body,
    });
  }
}
```

```
async cleanupExpired(db: DbExecutor, ttlHours: number): Promise<number> {  
  const cutoff = new Date(Date.now() - ttlHours * 60 * 60 * 1000);  
  return this.repo.deleteOlderThan(db, cutoff);  
}  
}
```



HTTP Layer

src/http/routes/accounts.routes.ts

159 lines · Account REST API

POST/GET /v1/accounts, balance, cursor-paginated entries. Thin handlers: Zod parse → service → JSON.

```
import type { FastifyPluginAsync } from 'fastify';
import { Buffer } from 'node:buffer';
import { AccountsRepository } from '../../../repositories/accounts.repo.js';
import { EntriesRepository } from '../../../repositories/entries.repo.js';
import { IdempotencyRepository } from '../../../repositories/idempotency.repo.js';
import { AccountsService } from '../../../services/accounts.service.js';
import { IdempotencyService } from '../../../services/idempotency.service.js';
import type { Database } from '../../../db/client.js';
import { createAccountRequestSchema } from '../schemas/accounts.schema.js';

export type AccountsRoutesOptions = {
  db: Database;
};

function encodeCursor(postedAt: Date, id: bigint): string {
  return Buffer.from(
    JSON.stringify({ postedAt: postedAt.toISOString(), id: id.toString() }),
    'utf8',
  ).toString('base64url');
}

function decodeCursor(cursor: string): { postedAt: Date; id: bigint } | undefined {
  try {
    const parsed = JSON.parse(Buffer.from(cursor, 'base64url').toString('utf8')) as {
      postedAt: string;
      id: string;
    };
    return { postedAt: new Date(parsed.postedAt), id: BigInt(parsed.id) };
  } catch {
    return undefined;
  }
}

export const accountsRoutes: FastifyPluginAsync<AccountsRoutesOptions> = async (fastify, opts) => {
  const accountsRepo = new AccountsRepository();
  const entriesRepo = new EntriesRepository();
  const idempotencyService = new IdempotencyService(new IdempotencyRepository());
  const service = new AccountsService(opts.db, accountsRepo, entriesRepo, idempotencyService);

  fastify.post(
    '/v1/accounts',
    {
      schema: {
        tags: ['accounts'],
        description: 'Create a ledger account',
        body: {
          type: 'object',
          required: ['name', 'type', 'currency'],
          properties: {
            name: { type: 'string', minLength: 1, maxLength: 100 },
            type: {
              type: 'string',
              enum: ['ASSET', 'LIABILITY', 'EQUITY', 'INCOME', 'EXPENSE'],
            },
            currency: { type: 'string', minLength: 3, maxLength: 3 },
          },
        },
      },
      response: {
        201: {
          type: 'object',
          properties: {
            id: { type: 'string', format: 'uuid' },
          },
        },
      },
    },
  );
};
```

```

        name: { type: 'string' },
        type: { type: 'string' },
        currency: { type: 'string' },
        status: { type: 'string' },
        created_at: { type: 'string', format: 'date-time' },
    },
    },
    },
    },
    },
    async (request, reply) => {
        if (reply.sent) {
            return;
        }

        const body = createAccountRequestSchema.parse(request.body);
        const response = await service.createAccount(
            { name: body.name, type: body.type, currency: body.currency },
            request.idempotency,
        );
        return reply.status(201).send(response);
    },
    );

fastify.get(
    '/v1/accounts/:id',
    {
        schema: {
            tags: ['accounts'],
            params: { type: 'object', properties: { id: { type: 'string', format: 'uuid' } } },
        },
    },
    async (request) => {
        const { id } = request.params as { id: string };
        const account = await service.getAccount(id);
        return service.toAccountResponse(account);
    },
);

fastify.get(
    '/v1/accounts/:id/balance',
    {
        schema: {
            tags: ['accounts'],
            params: { type: 'object', properties: { id: { type: 'string', format: 'uuid' } } },
        },
    },
    async (request) => {
        const { id } = request.params as { id: string };
        return service.getBalance(id);
    },
);

fastify.get(
    '/v1/accounts/:id/entries',
    {
        schema: {
            tags: ['accounts'],
            params: { type: 'object', properties: { id: { type: 'string', format: 'uuid' } } },
            querystring: {
                type: 'object',
                properties: {
                    limit: { type: 'integer', minimum: 1, maximum: 100, default: 50 },
                    cursor: { type: 'string' },
                },
            },
        },
    },
    async (request) => {
        const { id } = request.params as { id: string };
        const query = request.query as { limit?: number; cursor?: string };
        const limit = query.limit ?? 50;

        await service.getAccount(id);

        const cursor = query.cursor ? decodeCursor(query.cursor) : undefined;
        const rows = await entriesRepo.listByAccount(opts.db, id, limit, cursor);

        const entries = rows.map((e) => ({

```

```
        id: e.id.toString(),
        transaction_id: e.transactionId,
        account_id: e.accountId,
        direction: e.direction,
        amount: e.amount.toString(),
        currency: e.currency.trim(),
        posted_at: e.postedAt.toISOString(),
    }));

    const last = rows.at(-1);
    const next_cursor =
        rows.length === limit && last ? encodeCursor(last.postedAt, last.id) : null;

    return { entries, next_cursor };
},
);
};
```

POST /v1/transactions and GET by id. Delegates posting rules to TransactionsService.

```
import type { FastifyPluginAsync } from 'fastify';
import type { Database } from '../../db/client.js';
import { AccountsRepository } from '../../repositories/accounts.repo.js';
import { EntriesRepository } from '../../repositories/entries.repo.js';
import { IdempotencyRepository } from '../../repositories/idempotency.repo.js';
import { TransactionsRepository } from '../../repositories/transactions.repo.js';
import { IdempotencyService } from '../../services/idempotency.service.js';
import { TransactionsService } from '../../services/transactions.service.js';
import { postTransactionRequestSchema } from '../../schemas/transactions.schema.js';

export type TransactionsRoutesOptions = {
  db: Database;
};

export const transactionsRoutes: FastifyPluginAsync<TransactionsRoutesOptions> = async (
  fastify,
  opts,
) => {
  const service = new TransactionsService(
    opts.db,
    new AccountsRepository(),
    new EntriesRepository(),
    new TransactionsRepository(),
    new IdempotencyService(new IdempotencyRepository()),
  );

  fastify.post(
    '/v1/transactions',
    {
      schema: {
        tags: ['transactions'],
        description: 'Post a balanced double-entry transaction',
        body: {
          type: 'object',
          required: ['entries'],
          properties: {
            description: { type: 'string' },
            external_ref: { type: 'string' },
            entries: {
              type: 'array',
              minItems: 2,
              items: {
                type: 'object',
                required: ['account_id', 'direction', 'amount', 'currency'],
                properties: {
                  account_id: { type: 'string', format: 'uuid' },
                  direction: { type: 'string', enum: ['DEBIT', 'CREDIT'] },
                  amount: { type: 'string' },
                  currency: { type: 'string' },
                },
              },
            },
          },
        },
      },
    },
    async (request, reply) => {
      if (reply.sent) {
        return;
      }

      const body = postTransactionRequestSchema.parse(request.body);
      const entries = TransactionsService.parsePostingEntries(body.entries);
      const response = await service.postTransaction(
        {
          description: body.description,
          externalRef: body.external_ref,
          entries,
        },
        request.idempotency,
      );
    },
  );
}
```

```

        return reply.status(201).send(response);
    },
);

fastify.get(
  '/v1/transactions/:id',
  {
    schema: {
      tags: ['transactions'],
      params: { type: 'object', properties: { id: { type: 'string', format: 'uuid' } } },
    },
  },
  async (request) => {
    const { id } = request.params as { id: string };
    return service.getTransaction(id);
  },
);
};
};

```

src/http/routes/health.routes.ts

31 lines · Ops endpoints

/healthz liveness, /readyz DB check, /metrics Prometheus exposition with pool gauges.

```

import type { FastifyPluginAsync } from 'fastify';
import type pg from 'pg';
import { checkDbConnectivity } from '../../db/client.js';
import {
  getMetricsRegistry,
  getMetricsText,
  updatePoolMetrics,
} from '../../observability/metrics.js';

export type HealthRoutesOptions = {
  pool: pg.Pool;
};

export const healthRoutes: FastifyPluginAsync<HealthRoutesOptions> = async (fastify, opts) => {
  fastify.get('/healthz', async () => ({ status: 'ok' }));

  fastify.get('/readyz', async (_request, reply) => {
    const ok = await checkDbConnectivity(opts.pool);
    if (!ok) {
      return reply.status(503).send({ status: 'unavailable' });
    }
    return { status: 'ready' };
  });

  fastify.get('/metrics', async (_request, reply) => {
    updatePoolMetrics(opts.pool);
    const metrics = await getMetricsText();
    return reply.header('Content-Type', getMetricsRegistry().contentType).send(metrics);
  });
};

```


Requires header on POST, hashes body, returns cached response on replay, 409 on key/body mismatch. Business persist happens inside service transaction.

```
import type { FastifyPluginAsync } from 'fastify';
import type { Database } from '../db/client.js';
import { IdempotencyConflictError, IdempotencyKeyRequiredError } from '../domain/errors.js';
import { IdempotencyRepository } from '../repositories/idempotency.repo.js';
import {
  hashRequestBody,
  IdempotencyService,
  type IdempotencyContext,
} from '../services/idempotency.service.js';

export type IdempotencyPluginOptions = {
  db: Database;
};

export const idempotencyPlugin: FastifyPluginAsync<IdempotencyPluginOptions> = async (
  fastify,
  opts,
) => {
  const idempotencyService = new IdempotencyService(new IdempotencyRepository());

  fastify.addHook('preHandler', async (request, reply) => {
    if (request.method !== 'POST') {
      return;
    }

    const keyHeader = request.headers['idempotency-key'];
    const key = typeof keyHeader === 'string' ? keyHeader.trim() : undefined;
    if (!key || key.length === 0) {
      throw new IdempotencyKeyRequiredError();
    }

    const requestHash = hashRequestBody(request.body);
    const ctx: IdempotencyContext = { key, requestHash };
    request.idempotency = ctx;

    const lookup = await idempotencyService.findCached(opts.db, ctx);
    if (lookup === 'conflict') {
      throw new IdempotencyConflictError();
    }
    if (lookup) {
      request.idempotencyReplay = lookup;
      void reply.status(lookup.status).send(lookup.body);
    }
  });
};
```

Propagates request_id and trace_id into Pino child loggers; logs route, method, status, duration_ms on response.

```
import { randomUUID } from 'node:crypto';
import type { FastifyPluginAsync } from 'fastify';
import type { IdempotencyContext } from '../services/idempotency.service.js';

declare module 'fastify' {
  interface FastifyRequest {
    requestId: string;
    traceId: string;
    idempotency?: IdempotencyContext;
    idempotencyReplay?: { status: number; body: unknown };
  }
}

export const requestContextPlugin: FastifyPluginAsync = async (fastify) => {
  fastify.addHook('onRequest', async (request) => {
    const incoming =
      (typeof request.headers['x-request-id'] === 'string'
        ? request.headers['x-request-id']
        : undefined) ?? randomUUID();
    const trace =
      (typeof request.headers['x-trace-id'] === 'string'
        ? request.headers['x-trace-id']
        : undefined) ?? incoming;

    request.requestId = incoming;
    request.traceId = trace;
    request.log = request.log.child({
      request_id: incoming,
      trace_id: trace,
    });
  });

  fastify.addHook('onResponse', async (request, reply) => {
    const route = request.routeOptions.url ?? request.url;
    request.log.info({
      request_id: request.requestId,
      trace_id: request.traceId,
      route,
      method: request.method,
      status: reply.statusCode,
      duration_ms: reply.elapsedTime,
    });
  });
});
```

Maps DomainError and Zod failures to problem+json; surfaces Postgres double-entry violation messages without rewriting DB text.

```
import type { FastifyError, FastifyReply, FastifyRequest } from 'fastify';
import { ZodError } from 'zod';
import { DomainError, DoubleEntryViolationError } from '../domain/errors.js';

type ProblemDetails = {
  type: string;
  title: string;
  status: number;
  detail: string;
  instance?: string;
  code?: string;
};

function extractErrorMessage(err: unknown): string {
  if (!(err instanceof Error)) {
    return '';
  }
  let message = err.message;
  let cause: unknown = err.cause;
  while (cause instanceof Error) {
    message += ` ${cause.message}`;
    cause = cause.cause;
  }
  return message;
}

function isPostgresError(err: unknown): err is { code: string; message: string } {
  if (typeof err !== 'object' || err === null) {
    return false;
  }
  const record = err as Record<string, unknown>;
  return typeof record['code'] === 'string' && typeof record['message'] === 'string';
}

function mapDomainError(error: DomainError, instance: string): ProblemDetails {
  const base = {
    type: `https://ledger.local/problems/${error.code}`,
    detail: error.message,
    instance,
    code: error.code,
  };
  switch (error.code) {
    case 'account_not_found':
    case 'transaction_not_found':
      return { ...base, title: 'Not Found', status: 404 };
    case 'account_frozen':
    case 'idempotency_conflict':
      return { ...base, title: 'Conflict', status: 409 };
    case 'idempotency_key_required':
      return { ...base, title: 'Bad Request', status: 400 };
    case 'unbalanced_transaction':
    case 'currency_mismatch':
    case 'unsupported_currency':
    case 'invalid_amount':
    case 'double_entry_violation':
      return { ...base, title: 'Unprocessable Entity', status: 422 };
    default:
      return { ...base, title: 'Bad Request', status: 400 };
  }
}

export function errorHandler(
  error: FastifyError,
  request: FastifyRequest,
  reply: FastifyReply,
): void {
  const instance = request.url;
```

```

if (error instanceof DomainError) {
  const problem = mapDomainError(error, instance);
  void reply.status(problem.status).type('application/problem+json').send(problem);
  return;
}

if (error instanceof ZodError) {
  const problem: ProblemDetails = {
    type: 'https://ledger.local/problems/validation_error',
    title: 'Validation Error',
    status: 400,
    detail: error.message,
    instance,
    code: 'validation_error',
  };
  void reply.status(400).type('application/problem+json').send(problem);
  return;
}

const pgMessage = extractErrorMessage(error);
if (
  (isPostgresError(error) && error.message.includes('Double-entry violation')) ||
  pgMessage.includes('Double-entry violation')
) {
  const problem = mapDomainError(
    new DoubleEntryViolationError(pgMessage || error.message),
    instance,
  );
  void reply.status(problem.status).type('application/problem+json').send(problem);
  return;
}

if (error.validation) {
  const problem: ProblemDetails = {
    type: 'https://ledger.local/problems/validation_error',
    title: 'Validation Error',
    status: 400,
    detail: error.message,
    instance,
    code: 'validation_error',
  };
  void reply.status(400).type('application/problem+json').send(problem);
  return;
}

request.log.error({ err: error }, 'Unhandled error');
const problem: ProblemDetails = {
  type: 'https://ledger.local/problems/internal_error',
  title: 'Internal Server Error',
  status: 500,
  detail: 'An unexpected error occurred',
  instance,
  code: 'internal_error',
};
void reply.status(500).type('application/problem+json').send(problem);
}

```

Zod request/response shapes for accounts, balances, and statement entries — shared types for routes and tests.

```
import { z } from 'zod';

export const accountTypeSchema = z.enum(['ASSET', 'LIABILITY', 'EQUITY', 'INCOME', 'EXPENSE']);

export const accountStatusSchema = z.enum(['ACTIVE', 'FROZEN', 'CLOSED']);

export const createAccountRequestSchema = z.object({
  name: z.string().min(1).max(100),
  type: accountTypeSchema,
  currency: z.string().length(3),
});

export const accountResponseSchema = z.object({
  id: z.string().uuid(),
  name: z.string(),
  type: accountTypeSchema,
  currency: z.string().length(3),
  status: accountStatusSchema,
  created_at: z.string().datetime(),
});

export const balanceResponseSchema = z.object({
  account_id: z.string().uuid(),
  currency: z.string().length(3),
  balance: z.string(),
  journal_balance: z.string(),
  as_of: z.string().datetime(),
});

export const entryStatementSchema = z.object({
  id: z.string(),
  transaction_id: z.string().uuid(),
  account_id: z.string().uuid(),
  direction: z.enum(['DEBIT', 'CREDIT']),
  amount: z.string(),
  currency: z.string().length(3),
  posted_at: z.string().datetime(),
});

export const entriesListResponseSchema = z.object({
  entries: z.array(entryStatementSchema),
  next_cursor: z.string().nullable(),
});

export type CreateAccountRequest = z.infer<typeof createAccountRequestSchema>;
export type AccountResponse = z.infer<typeof accountResponseSchema>;
```

Zod schemas for posting payloads (amount as string) and transaction responses.

```
import { z } from 'zod';

export const entryInputSchema = z.object({
  account_id: z.string().uuid(),
  direction: z.enum(['DEBIT', 'CREDIT']),
  amount: z.string().regex(/^\d+$/),
  currency: z.string().length(3),
});

export const postTransactionRequestSchema = z.object({
  description: z.string().max(500).optional(),
  external_ref: z.string().max(200).optional(),
  entries: z.array(entryInputSchema).min(2),
});

export const entryResponseSchema = z.object({
  id: z.string(),
  transaction_id: z.string().uuid(),
  account_id: z.string().uuid(),
  direction: z.enum(['DEBIT', 'CREDIT']),
  amount: z.string(),
  currency: z.string().length(3),
  posted_at: z.string().datetime(),
});

export const transactionResponseSchema = z.object({
  id: z.string().uuid(),
  status: z.enum(['POSTED', 'REVERSED']),
  description: z.string().nullable(),
  external_ref: z.string().nullable(),
  posted_at: z.string().datetime(),
  entries: z.array(entryResponseSchema),
});

export type PostTransactionRequest = z.infer<typeof postTransactionRequestSchema>;
```



Jobs

src/jobs/balance-verification.ts

103 lines · Balance reconciliation

Compares materialised account_balances to journal-derived totals per account. CLI via npm run verify-balances; optional interval scheduler; emits ledger_balance_drift_total on mismatch.

```
import type { Database } from '../db/client.js';
import { createDbContext } from '../db/client.js';
import { loadConfig } from '../config.js';
import { balanceFromJournal, mapAccountRow } from '../domain/account.js';
import { getLogger } from '../observability/logger.js';
import { ledgerBalanceDriftTotal } from '../observability/metrics.js';
import { AccountsRepository } from '../repositories/accounts.repo.js';
import { EntriesRepository } from '../repositories/entries.repo.js';

export type BalanceDrift = {
  accountId: string;
  materialised: string;
  journal: string;
  delta: string;
};

export type BalanceVerificationReport = {
  checkedAt: string;
  accountsChecked: number;
  drifts: BalanceDrift[];
};

export async function verifyBalances(db: Database): Promise<BalanceVerificationReport> {
  const accountsRepo = new AccountsRepository();
  const entriesRepo = new EntriesRepository();
  const logger = getLogger();

  const accountIds = await accountsRepo.listAllAccountIds(db);
  const drifts: BalanceDrift[] = [];

  for (const accountId of accountIds) {
    const accountRow = await accountsRepo.findById(db, accountId);
    if (!accountRow) {
      continue;
    }
    const account = mapAccountRow(accountRow);
    const balanceRow = await accountsRepo.getBalance(db, accountId);
    if (!balanceRow) {
      continue;
    }

    const { debits, credits } = await entriesRepo.sumJournalByAccount(db, accountId);
    const journalBalance = balanceFromJournal(account.type, debits, credits);
    const materialised = balanceRow.balance;

    if (journalBalance !== materialised) {
      drifts.push({
        accountId,
        materialised: materialised.toString(),
        journal: journalBalance.toString(),
        delta: (materialised - journalBalance).toString(),
      });
    }
  }

  if (drifts.length > 0) {
    ledgerBalanceDriftTotal.inc(drifts.length);
    logger.error({ drifts, count: drifts.length }, 'Balance drift detected');
  } else {
    logger.info({ accountsChecked: accountIds.length }, 'Balance verification passed');
  }
}
```

```

    return {
      checkedAt: new Date().toISOString(),
      accountsChecked: accountIds.length,
      drifts,
    };
  }
}

async function runCli(): Promise<void> {
  const config = loadConfig();
  const { db, pool } = createDbContext(config);
  try {
    const report = await verifyBalances(db);
    console.log(JSON.stringify(report, null, 2));
    if (report.drifts.length > 0) {
      process.exit(1);
    }
  } finally {
    await pool.end();
  }
}

const isCli = process.argv[1]?.includes('balance-verification') && process.argv.includes('--cli');
if (isCli) {
  runCli().catch((err: unknown) => {
    console.error(err);
    process.exit(1);
  });
}

export function startBalanceVerificationScheduler(
  db: Database,
  intervalMs: number,
): NodeJS.Timeout | undefined {
  if (intervalMs <= 0) {
    return undefined;
  }
  return setInterval(() => {
    void verifyBalances(db);
  }, intervalMs);
}

```




Observability

src/observability/logger.ts

36 lines · Structured logging

Pino with JSON in production, pretty in dev. Redacts Idempotency-Key and auth headers from logs.

```
import pino from 'pino';
import type { AppConfig } from '../config.js';

let rootLogger: pino.Logger | undefined;

export function createLogger(config: AppConfig): pino.Logger {
  const isDev = config.nodeEnv === 'development';
  const options: pino.LoggerOptions = {
    level: config.logLevel,
    redact: {
      paths: [
        'req.headers.authorization',
        'req.headers["idempotency-key"]',
        'req.headers.idempotency-key',
        'headers.authorization',
        'headers["idempotency-key"]',
        'headers.idempotency-key',
      ],
      censor: '[REDACTED]',
    },
  };
  if (isDev) {
    options.transport = {
      target: 'pino-pretty',
      options: { colorize: true, translateTime: 'SYS:standard' },
    };
  }
  rootLogger = pino(options);
  return rootLogger;
}

export function getLogger(): pino.Logger {
  rootLogger ??= pino({ level: 'info' });
  return rootLogger;
}
```

http_requests_total, http_request_duration_seconds, ledger_* counters, db_pool_* gauges for operability dashboards.

```
import { Counter, Gauge, Histogram, Registry, collectDefaultMetrics } from 'prom-client';
import type pg from 'pg';

const register = new Registry();
collectDefaultMetrics({ register });

export const httpRequestsTotal = new Counter({
  name: 'http_requests_total',
  help: 'Total HTTP requests',
  labelNames: ['method', 'route', 'status'] as const,
  registers: [register],
});

export const httpRequestDurationSeconds = new Histogram({
  name: 'http_request_duration_seconds',
  help: 'HTTP request duration in seconds',
  labelNames: ['method', 'route', 'status'] as const,
  buckets: [0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10],
  registers: [register],
});

export const ledgerTransactionsPostedTotal = new Counter({
  name: 'ledger_transactions_posted_total',
  help: 'Total ledger transactions posted',
  registers: [register],
});

export const ledgerEntriesPostedTotal = new Counter({
  name: 'ledger_entries_posted_total',
  help: 'Total ledger entries posted',
  registers: [register],
});

export const ledgerBalanceDriftTotal = new Counter({
  name: 'ledger_balance_drift_total',
  help: 'Accounts where materialised balance diverges from journal',
  registers: [register],
});

const poolSizeGauge = new Gauge({
  name: 'db_pool_size',
  help: 'Database pool total connections',
  registers: [register],
});

const poolIdleGauge = new Gauge({
  name: 'db_pool_idle',
  help: 'Database pool idle connections',
  registers: [register],
});

const poolWaitingGauge = new Gauge({
  name: 'db_pool_waiting',
  help: 'Database pool waiting requests',
  registers: [register],
});

export function updatePoolMetrics(pool: pg.Pool): void {
  poolSizeGauge.set(pool.totalCount);
  poolIdleGauge.set(pool.idleCount);
  poolWaitingGauge.set(pool.waitingCount);
}

export function getMetricsRegistry(): Registry {
  return register;
}

export async function getMetricsText(): Promise<string> {
```

```
return register.metrics();  
}
```

